1. **True/False Questions** (20% with 2% each question)

Please indicate whether each of the following statements is true or false.

(a) For a given Markov Decision Process (MDP), there is a unique set of state values $v_\pi(s)$ and action values $q_\pi(s,a)$ for all states s and actions a for an action policy π. True or False? Answer: True

(b) Bellman expectation and optimality equations are valid and applicable to a system regardless of whether it satisfies the Markovian property or not. True or False? Answer: False

(c) Without knowing the model and by using the experience data, the policy-gradient approach to reinforcement learning effectively finds the optimal action policy for the corresponding MDP for the environment. True or False? Answer: False

(d) Q-learning is more "aggressive" than the SARSA technique in updating the Q-values and therefore Q-learning may suffer more than SARSA from the bias effects of the Q-value estimations. True or False? Answer: True

(e) The essence of model-free reinforcement learning is that the experience data samples the system, without a need to know the model of the system. True or False? Answer: True

(f) The system must satisfy the memoryless property for the Monte Carlo (MC) or Temporal Difference (TD) techniques to obtain the exact state or action value functions. True or False? Answer: True

(g) The purpose of replay buffer is to support use of random samples from the collected data stored in the buffer to improve the process of Q-learning without causing an excessive amount of computation. True or False? Answer: True

(h) Temporal difference (TD) learning by using neural network as approximation can always converge to the optimal action policy. True or False? Answer: False

(i) Forward and backward TD learning are equivalent because both the forward and backward approaches update the Q-values for the same total amount during the training process. True or False? Answer: True

(j) When updating the model parameters, policy-gradient approach tends to increase the probability of taking actions that lead to improved return. True or False? Answer: True

2. **Dynamic Programming** (25%)

A Markov Decision Process (MDP) with three states {A, B, C} and two actions {i, j} is presented in Figure 1. The actions, state transitions and rewards are also shown in the figure. For example, when action i is taken in state A, the system state transits to B or back to A with probability 0.5 and 0.5, respectively. If action j is taken while in state A, the state is changed to C with probability 0.2 and the state remains unchanged with probability 0.8. The reward is given according to the current state. For example, if the state is changed to A, a reward of +2 will be given no matter what the pervious state is. The discount factor is $\gamma = 0.9$. The return for any

given state s is the total of discounted rewards over time starting from state s by following an action policy π.

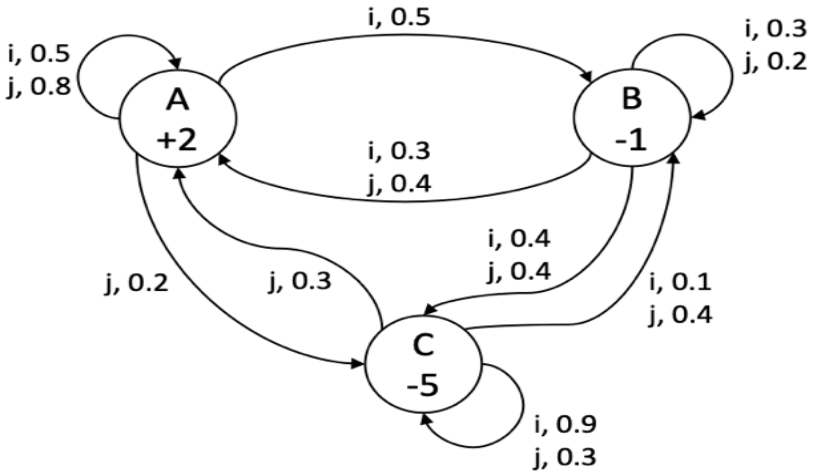


Figure 1. A simple Markov Decision Process (MDP)

(a) Why is it called a "Markov" decision process? [2]

Answer: The process satisfies the Markovian (memoryless) property. That is, the probability of transition from state s to s' is determined only by the current state s and current action a and not any other pervious states or actions.

(b) Assume all state values are initialized to zero. Fill out the following table with the results of value iteration and γ=0.9. [6]

k	$v_k(\mathrm{A})$	$v_k(B)$	$v_k(C)$
1			
2			

Answer:

k	$v_k(\mathrm{A})$	$v_k(B)$	$v_k(C)$
1	2	-1	-5
2	2.54	-2.26	-6.17

$v_2(\mathrm{A})$ = max { 2+0.9*(0.5*2+0.5*-1), 2+0.9*(0.8*2+0.2*-5) } = 2.54

$v_2(\mathrm{B})$ = max { -1+0.9*(0.3*2+0.3*-1+0.4*-5), -1+0.9*(0.4*2+0.2*-1+0.4*-5) } = -2.26

$v_2(\mathrm{B})$ = max { -5+0.9*(0.1*-1+0.9*-5), -5+0.9*(0.3*2+0.4*-1+0.3*-5) } = -6.17

(c) For k=2, what action would you take given state A? Is this action policy optimal? [6]

Answer: q(A, i) = 0.5*(2+$v_2(\mathrm{A})$) + 0.5*(-1+$v_2(\mathrm{B})$) = 0.64

q(A, j) = 0.8*(2+$v_2(\mathrm{A})$) + 0.2*(-5+$v_2(\mathrm{B})$) = 1.398

By comparing the above action values, one would take action *j* when in the state A. Since the value iteration has not converged when k=2, any action policy based on the current action values is not necessarily optimal.

(d) Let $v_k(\mathrm{s})$ denote the value of state s in iteration k of (synchronous) value iteration. Let us define $f_k(s,a) \equiv r + \gamma \sum_{s'} P(s'|s,a)\, v_k(s')$ where r is the immediate reward by action $a \in A$ in state s and $P(s'|s,a)$ is the transition probability to next state s' given current state s and action a. What is the most restrictive relationship between $v_{k+1}(\mathrm{s})$ and $f_k(s,a)$ for any $a \in A$? Please use mathematical expressions to show your result. The most restrictive relationship means that, for example, if x < y always holds, we use < instead of ≤. [5]

Answer: We have $v_{k+1}(s) \geq f_k(s,a) = r + \gamma \sum_{s'} P(s'|s,a)\, v_k(s')$ because the Bellman optimality equation gives us

$$v_{k+1}(s) = \max_{a \in A} \{ r + \gamma \sum_{s'} P(s'|s,a)\, v_k(s') \}$$

$$\geq r + \gamma \sum_{s'} P(s'|s,a)\, v_k(s') \quad \text{for any } a \in A.$$

$$= f_k(s,a).$$

(e) Consider the order of computational complexity of value iteration for the following two variants of the problem: (A) Keep the action space unchanged {i, j}, and add two states D and F and associated state transitions to the MDP, and (B) Keep the state space unchanged, and add two actions {h, g} and associated state transitions. Which one of the two variants, A and B, has higher computational complexity? Briefly explain your result. [6]

Answer: The computational complexity for value iteration per iteration is $O(mn^2)$ for an MDP with *m* actions and *n* states. Thus, adding two states (Variant A) typically increases computational complexity more than that by adding two actions (Variant B).

3. Model-free Algorithms (20%)

Figure 2 shows a grid world with the following characteristics:

- Four states {A, B, C, D} and a terminal state T on both ends.
- Actions from state A, B, C, and D are Left and Right, which will move the agent to the adjacent left or right position.
- The reward for reaching a terminal state T is 10. Otherwise, the reward is -1 for each time step.

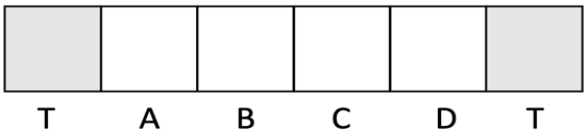


Figure 2: A grid world.

(a) Explain why Q-learning is off-policy and SARSA is on-policy. [2]

Answer: As reflected in the update formula, Q-learning always tries to identify the optimal action based on the current Q-value estimates. This target (greedy) policy is not necessarily the policy used in the environment. So, Q-learning is off-policy. In contrast, the SARSA update formula considers the actions as taken by the behavior policy used in the environment. Therefore, SARSA is on-policy.

(b) State two advantages of using value function approximation compared to a tabular representation for reinforcement learning. [2]

Answer: i) Generalizability to unseen states and ii) more efficient memory usage (i.e., more compact value function representation).

(c) Briefly describe "bootstrapping" in the context of reinforcement learning and discuss whether Monte-Carlo method requires bootstrapping. [3]

Answer: Bootstrapping is the process of updating value estimates of states (or state/action pairs) based on value estimates of other states (or state/action pairs). Monte-Carlo method does not require bootstrapping because it estimates the value function based on complete episodes after reaching terminal states, and updates values from actual returns from episodic data.

(d) For the grid world in Figure 2, let the agent begin in state C and four complete or incomplete episodes are collected as follows:

C, Left, -1, B, Left, -1, A, Right, -1, B
A, Left, 10, T
D, Left, -1, C, Left, -1, B, Right, -1, C
C, Right, -1, D, Right, 10, T

Can we apply the first-visit Monte-Carlo (MC) method to estimate the state-value functions of V(B) and V(C) from the episodes with the discount factor of $\gamma = 1$? If so, compute the estimated values of V(B) and V(C) by using the first-visit MC. Otherwise, briefly explain the reasons. [8]

Answer: Monte-Carlo methods typically require full episodes (after reaching terminal state) to estimate the value function from the complete returns (i.e., the total accumulated rewards from an initial state to the terminal state). Therefore, for the four episodes without reaching a terminal state, other methods like TD learning are more suitable due to their ability to learn from incomplete sequences. We can use the second and last episodes to estimate the value function by MC method. Since state B is not included in these two complete episodes, we can only estimate the V(C) as

V(C) = -1+10 = 9.

(e) Assume that the SARSA algorithm is used to update the Q values with $\gamma = 1$, $\alpha = 1$, and the agent taking the sequence of actions {Right, Left, Left} from state A. Fill the following table of Q values after these updates. [5]

Q(s,a)	A	B	C	D
Right				
Left				

Answer:

Q(s,a)	A	B	C	D
Right	-1	0	0	0
Left	10	-1	0	0

4. RL Scheduler for Uplink Transmissions (30%)

Consider three neighboring cells *A*, *B* and *C* in a cellular wireless network in Figure 3, each of which has a single base station serving a set of *N* users (terminals) denoted by G_A, G_B, and G_C in the cells, respectively. We consider here only the "uplink transmissions," which correspond to data transmission from users to their serving base stations.

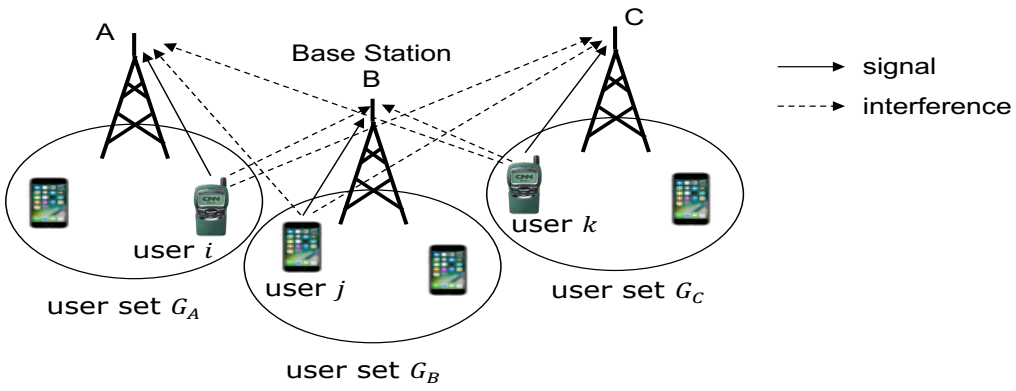


Figure 3. Uplink transmissions in three neighboring cells *A*, *B* and *C*.

The radio channel is divided into N time slots where each user in every cell is scheduled to transmit exactly once in one of the time slots. A schedule for the uplink transmissions is given in Figure 4 where users $i_t \in G_A$, $j_t \in G_B$ and $k_t \in G_C$ in cell A, B and C, respectively, are scheduled to transmit concurrently in each time slot $t = 1$ to N.

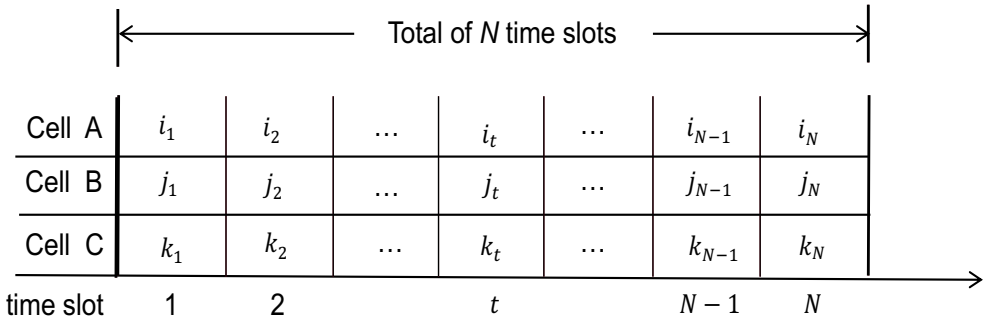


Figure 4. Schedule for uplink transmissions for all three cells in N time slots.

Due to interference and other factors, the data rate of each uplink transmission is limited to achieve satisfactory reception. When users i_t, j_t and k_t in the three cells are transmitting simultaneously in time slot $t = 1$ to N, let $R_A(i_t, j_t, k_t)$, $R_B(i_t, j_t, k_t)$ and $R_C(i_t, j_t, k_t)$ be the data rate for the uplink transmission from each of the three users in cell A, B and C, respectively. Clearly, which users are scheduled to transmit simultaneously affects the data rates of all transmissions in the same time slot.

Our goal is to develop a reinforcement-learning algorithm (agent) to schedule the uplink transmissions to maximize the total data rate by the given sets of users in all three cells over N time slots. Let us focus on an algorithm that selects and schedules one user in each of the three cells, who have not transmitted earlier, to transmit in time slot t starting from $t = 1$ to N.

(a) Define the system state S_t at the beginning of time slot t (i.e., before the agent schedules 3 users for transmission in the time slot) and briefly explain your result. [8]

Answer: Let $S_t = (S_t^A, S_t^B, S_t^C)$ where S_t^i denotes the set of remaining users in cell i for $i = A, B$ or C who will be scheduled for transmission from time slot t until N. We initialize $S_1^A = G_A$, $S_1^B = G_B$, and $S_1^C = G_C$. This state definition is appropriate because it records the remaining users who need to be scheduled for transmission in time slot t and onward.

(b) By using the notation above, define the action a_t in time slot t. [4]

Answer: By using the notation above, we define $A_t \equiv (i_t, j_t, k_t)$ where users i_t, j_t and k_t in cell A, B and C, respectively, are scheduled to transmit in time slot t.

(c) Using the notation above, what is the immediate reward R_{t+1} for the system state S_t and action A_t taken by the agent in time slot t? [3]

Answer: The immediate reward is the sum of data rates for the concurrent transmissions in time slot t. That is, $R_{t+1} = R_A(i_t, j_t, k_t) + R_B(i_t, j_t, k_t) + R_C(i_t, j_t, k_t)$.

(d) How does the system state change from S_t to S_{t+1} for action A_t taken by the agent? [4]

Answer: The next state is $S_{t+1} = (S_{t+1}^A, S_{t+1}^B, S_{t+1}^C)$ where $S_{t+1}^A = S_t^A - \{i_t\}$, $S_{t+1}^B = S_t^B - \{j_t\}$ $S_{t+1}^C = S_t^C - \{k_t\}$. Note that the state transition is deterministic for a given action A_t.

(e) For the objective of maximizing the total data rate for all transmissions in all N time slots, what is the reward discount factor γ and why? [3]

Answer: The discount factor is $\gamma = 1$ because the date rates for all time slots are treated equally.

(f) If Monte Carlo method is used to train the agent, provide the steps involved in the method and identify how the optimal transmission schedule can be obtained. [8]

Answer: The Monte Carlo method consists of the following steps.

i) Initialize the visit counts $N(S_t, A_t) \leftarrow 0$ and the state-action function $Q(S_t, A_t) \leftarrow 0$ for all state S_t and action A_t, and set $k = 1$.

ii) Choose an arbitrary policy π (such as a random policy).

iii) Generate the k-th transmission schedule (e.g., k-th episode of data) using the policy π. Let this episode be denoted by $S_1, A_1, R_2, S_2, A_2, R_3, \ldots, S_N, R_{N+1}$.

iv) For each state S_t and action A_t in the episode, update the following:

$$H_t \leftarrow \sum_{\tau=t}^{N} R_{\tau+1}$$

$$N(S_t, A_t) \leftarrow N(S_t, A_t) + 1$$

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)}(H_t - Q(S_t, A_t)) \qquad (*)$$

v) Form an ϵ-greedy policy π' based on the updated action-value function from Step iv) and setting $k \leftarrow k + 1$ and $\epsilon = \frac{1}{k}$.

vi) If the update equation (*) indicates a relatively small amount of update for $Q(S_t, A_t)$ for all S_t and A_t, the method converges and thus stops. Otherwise, repeat from Step iii) with policy π replaced by policy π'.

When the method converges, the optimal policy is obtained by

$$\pi^*(a|s) = \lim_{k \to \infty} 1(a = \arg\max_{a' \in A} Q_k(s, a')) \quad \text{for every state } s.$$